

EVALUATION METRICS for RAG

How to know if your RAG system actually works
not just feels like it works

 Retrieve →  Generate →  Evaluate



AI Coach John

Follow



PROITBRIDGE
DRIVE FOR BETTER FUTURE

 Repost

What is RAG Evaluation & Why It Matters

"It works on my test queries" is not evaluation. It's vibes.

RAG has two failure points: bad retrieval (wrong docs) or bad generation (right docs, wrong answer)

Without metrics, you can't tell which one is broken or if both are.

⚠️ REAL SCENARIO

Your support bot gives a wrong answer about refund policy.

- Is it because retrieval pulled the wrong document?
- Or because generation hallucinated despite having the right document?

Without evaluation, you're debugging blind.

📁 2 EVALUATION LAYERS

- **Retrieval Layer** → Did we fetch the right documents?
- **Generation Layer** → Did the LLM use those documents correctly to answer?

Evaluate both separately. A great retriever + bad generator (or vice versa) still gives a broken system.



Retrieval Layer Overview

CONCEPT

Retrieval Layer = "Did we even give the LLM the right material?"

- > This layer is evaluated before generation happens
- > If this layer fails, fixing prompts or switching models won't help
- > Fixes here usually involve: chunking strategy, embedding model, top-k value, or reranking

SCENARIO

User asks "What's the cancellation policy for premium plans?"

Your retriever returns 5 chunks but the actual policy is in chunk 6. Generation never had a chance, no matter how good the LLM is.

4 KEY RETRIEVAL METRICS

- > **Context Precision**
- > **Context Recall**
- > **MRR** (Mean Reciprocal Rank)
- > **Hit Rate**



Context Precision

🔗 WHAT IT MEASURES

Of all the chunks retrieved, how many are actually relevant to the question?

🔍 SCENARIO

Query: "What's the refund policy for premium plans?"

Retriever returns 5 chunks only 2 are about refund policy, 3 are about unrelated billing topics

Context Precision = $2/5 = 0.4$ (low)

⚠️ WHY IT MATTERS

- > Low precision = noisy context LLM has to "search" through irrelevant text
- > More noise = higher chance of the LLM picking up wrong details or getting distracted
- > Often caused by: too large top-k, poor chunk boundaries, weak embeddings

🔧 HOW TO IMPROVE

- > Reduce top-k
- > Add a reranker after initial retrieval
- > Improve chunking smaller, more focused chunks



Context Recall

⚙️ WHAT IT MEASURES

Of all the chunks that are actually relevant in your knowledge base, how many did the retriever successfully fetch?

🔍 SCENARIO

Your KB has 3 chunks relevant to "premium plan cancellation" refund %, cancellation window, and exceptions

Retriever only fetches 1 of these 3

Context Recall = $1/3 = 0.33$ (low)

⚠️ WHY IT MATTERS

- > Low recall = LLM is answering with incomplete information
- > Even a perfect LLM can't generate a complete answer if half the facts were never retrieved
- > Caused by: weak embeddings, chunks too large/small, missing docs

🔧 HOW TO IMPROVE

- > Increase top-k (with reranking to control precision)
- > Try hybrid search (keyword + semantic)
- > Re-chunk documents with better overlap



MRR

(Mean Reciprocal Rank)

⚙️ WHAT IT MEASURES

How high up in the ranked results does the first relevant chunk appear?

🔍 SCENARIO

Query: "What's the cancellation policy?"

Case A: Relevant chunk is ranked #1 →
 $RR = 1/1 = 1.0$

Case B: Relevant chunk is ranked #3 →
 $RR = 1/3 = 0.33$

MRR = average of reciprocal ranks across all test queries

💡 WHY IT MATTERS

- Even if recall is fine, if the relevant chunk is buried at pos 8, the LLM might not even "see" it
- High MRR = your ranking surfaces the right answer early
- Critical when only passing top-3 or top-5 chunks to LLM

🔧 HOW TO IMPROVE

- Add a reranker (e.g. cross-encoder)
- Fine-tune embeddings on domain-specific data
- Use hybrid retrieval to boost exact-match terms



Hit Rate

⚙️ WHAT IT MEASURES

Across all test queries, what % of the time does at least ONE relevant chunk appear in the top-k results?

🔍 SCENARIO

You run 100 test queries through your retriever with top-k = 5

For 88 queries, at least one relevant chunk was in the top 5

For 12 queries, none of the top 5 chunks were relevant

Hit Rate = $88/100 = 0.88$

⚠️ WHY IT MATTERS

- > This is your "baseline sanity check" if hit rate is low, nothing downstream can be fixed by prompting
- > A low hit rate on specific query types often points to a documentation gap, not a retrieval bug

🔧 HOW TO IMPROVE

- > Audit failing queries is the info even in your KB?
- > Add missing documents/FAQs
- > Try different embedding models



Generation Layer Overview

CONCEPT

Generation Layer = "Given the right material, did the LLM use it correctly?"

- > This layer is evaluated after retrieval assumes context is already correct
- > Fixes here usually involve: prompt engineering, model choice, temperature settings, instruction tuning

SCENARIO

Retrieval worked perfectly correct
policy doc was in context.

But the LLM still says "Premium plans can be cancelled anytime with full refund" when the doc says "50% refund within 30 days."

That's a generation failure:
hallucination despite correct context.

4 KEY GENERATION METRICS

- > Faithfulness
- > Answer Relevancy
- > Answer Correctness
- > Context Utilization



Faithfulness (No Hallucination)

⚙️ WHAT IT MEASURES

Is the generated answer factually grounded in the retrieved context or did the LLM make things up?

🔍 SCENARIO

Context: "Premium plans offer a 50% refund if cancelled within 30 days."

LLM answer: "Premium plans can be cancelled anytime for a full refund."

The answer contradicts the context entirely → **Faithfulness = low**

💡 WHY IT MATTERS / HOW CALC.

- > Break the answer into individual claims, check each claim against context
- > This is the #1 metric for catching hallucinations
- > Low faithfulness = your prompt isn't enforcing grounding

🔧 HOW TO IMPROVE

- > Add explicit instructions: "Only answer based on the provided context."
- > Lower temperature
- > Try a model with stronger instruction-following



Answer Relevancy

⚙️ WHAT IT MEASURES

Does the generated answer actually address what the user asked or does it go off-topic?

🔍 SCENARIO

Query: "What's the cancellation policy for premium plans?"

LLM answer: "Premium plans include priority support, advanced analytics, and custom branding options."

Factually correct but off-topic →
Relevancy = low

⚠️ WHY IT MATTERS / HOW CALC.

- > High faithfulness + low relevancy = "technically true, but useless" answers
- > Common cause: retrieved context was about a related-but-wrong topic, and LLM just summarized whatever it got

🔧 HOW TO IMPROVE

- > Improve retrieval precision (often the root cause)
- > Add instruction: "If the context doesn't directly answer the question, state that clearly"



Answer Correctness

⚙️ WHAT IT MEASURES

Is the generated answer factually correct when compared against a ground truth (human-verified) answer?

🔍 SCENARIO

Ground truth: "Premium plans offer 50% refund if cancelled within 30 days."

LLM answer: "Premium plans offer a full refund if cancelled within 30 days."

Faithful & relevant, but wrong →

Correctness = low

⚠️ WHY IT MATTERS / HOW CALC.

- > Combines semantic similarity with factual overlap
- > Requires a ground truth dataset most dependent on good test set preparation
- > Closest metric to "is this answer actually right"

🔧 HOW TO IMPROVE

- > Audit and update source documents regularly
- > Use this metric specifically to catch stale/incorrect knowledge base content



Context Utilization

⚙️ WHAT IT MEASURES

Did the LLM actually use the relevant parts of the retrieved context, or did it ignore them?

🔍 SCENARIO

Context contains the correct policy:
"50% refund within 30 days"

LLM answer: "Typically, premium plans offer refunds within a standard cancellation window" (vague, generic)

Context Utilization = low

⚠️ WHY IT MATTERS / HOW CALC.

- Catches a subtle failure mode: the LLM "defaults" to general knowledge instead of using your specific context
- Dangerous for domain-specific or company-specific info generic LLM knowledge may be outdated

🔧 HOW TO IMPROVE

- Strengthen prompt: "Base your answer strictly on the information in the context below..."
- Place retrieved context closer to the question in the prompt



RAGAS Test Set Prep

RAGAS is the most widely used open-source framework covers both retrieval and generation metrics out of the box.

4 FIELDS PER EXAMPLE

- **question** the user query
- **contexts** chunks retrieved by your RAG pipeline
- **answer** your RAG system's generated response
- **ground_truth** the correct, human-verified answer

BUILDING A REAL TEST SET

- Sample real user queries from production logs (20-50 min)
- Run each through your live RAG pipeline to capture actual contexts and answer
- Have a domain expert write ground_truth for each this is key

</> PYTHON DATA STRUCTURE

```
test_data = {
    "question": [
        "What is the cancellation policy?",
        "How long does shipping take?"
    ],
    "contexts": [
        ["Premium plans cancelled within 30 days for a 50% refund."],
        ["Shipping takes 7-14 business days depending on location."]
    ],
    "answer": [
        "You can cancel anytime for a full refund.",
        "Shipping takes 7-14 business days."
    ],
    "ground_truth": [
        "Premium plans offer 50% refund if cancelled within 30 days.",
        "Orders take 7 to 14 business days to ship."
    ]
}
```



RAGAS Dataset Format

↩ CONVERSION STEP

Convert your test data into RAGAS-compatible format

```
from datasets import Dataset

eval_dataset =
Dataset.from_dict(test_data)
```

⚙ WHAT THIS DOES

- Converts the dictionary into a Hugging Face Dataset object
- RAGAS metrics are built to operate on this format handles batching internally

💡 TIPS FOR PRODUCTION

- Keep a separate test set per "topic area" (billing, shipping, technical support) helps you pinpoint where issues cluster
- Re-run evaluation after every major change model upgrade, prompt change, embedding model swap, chunking strategy change
- Version your test sets track how scores change over time as your RAG system evolves



RAGAS Evaluation

>_ RUNNING THE EVALUATION

Run evaluation across both layers in a single call:

```
from ragas import evaluate
from ragas.metrics import (
    context_precision,
    context_recall,
    faithfulness,
    answer_relevancy
)

result = evaluate(
    dataset=eval_dataset,
    metrics=[
        context_precision, # Retrieval layer
        context_recall,     # Retrieval layer
        faithfulness,       # Generation layer
        answer_relevancy    # Generation layer
    ]
)

print(result)
```

▮ SAMPLE OUTPUT

```
{
  'context_precision': 0.85,
  'context_recall': 0.78,
  'faithfulness': 0.62,
  'answer_relevancy': 0.91
}
```



Reading Results & Taking Action

INTERPRETING THE SAMPLE OUTPUT

METRIC	SCORE	DIAGNOSIS
Context Precision	0.85	Retrieval is fairly clean minimal noise
Context Recall	0.78	Mostly good some relevant info still missed
Faithfulness	0.62	⚠️ LLM is hallucinating despite decent context
Answer Relevancy	0.91	Answers are on-topic

🔍 ROOT CAUSE ANALYSIS

- Retrieval layer is performing reasonably not the main issue
- Generation layer has a faithfulness problem the LLM isn't sticking to retrieved facts

🛠️ ACTION PLAN

- Add stricter grounding instructions to the prompt
- Lower temperature for more deterministic, less "creative" outputs
- Re-run evaluation faithfulness should rise without hurting relevancy

🔄 Evaluation Loop: Measure → Diagnose → Fix → Re-measure



AI Coach John

Follow



PROTIBRIDGE
DRIVER FOR BETTER LEARNERS

🔄 Repost

Evaluation Cheatsheet

LAYER	METRIC	TELLS YOU
🔍 Retrieval	Context Precision	Are retrieved chunks relevant?
🔍 Retrieval	Context Recall	Did we miss relevant chunks?
🔍 Retrieval	MRR	Is the right chunk ranked high?
🔍 Retrieval	Hit Rate	Is relevant info in top-k at all?
⚙️ Generation	Faithfulness	Is the answer grounded, not hallucinated?
⚙️ Generation	Answer Relevancy	Does it answer the actual question?
⚙️ Generation	Answer Correctness	Is it factually right vs ground truth?
⚙️ Generation	Context Utilization	Did the LLM use the specific context given?

🏗️ Framework: RAGAS

Open-source, plug-and-play with LangChain & LlamaIndex

